C++ COMPLETE REFERENCE LIBRARY • MODULE 21

21 STL AND MODERN CPP — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 stl architecture and sequence containers
3. **Theory Chapter:** 02 associative containers map and set
4. **Theory Chapter:** 03 iterators algorithms and modern cpp23
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (3 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

21 — Standard Template Library (STL) and Modern C++23

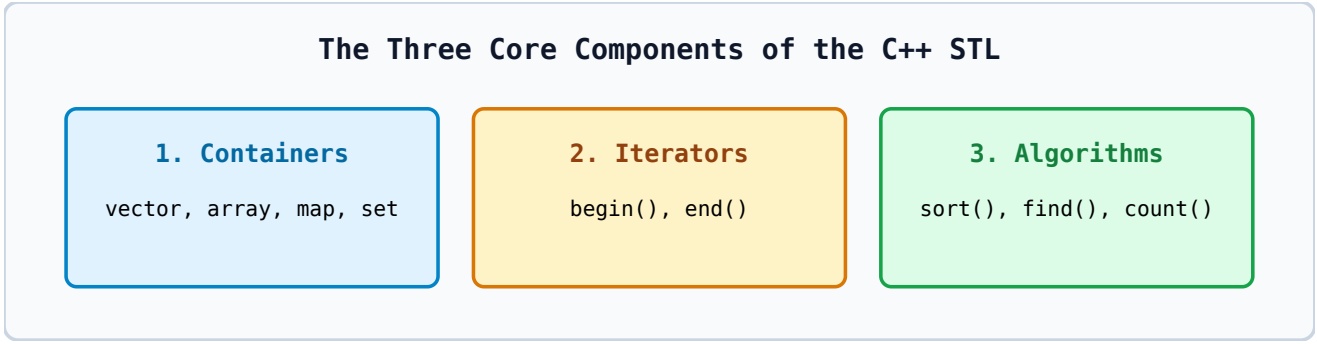
This module introduces the three core components of the C++ Standard Template Library (**Containers**, **Iterators**, and **Algorithms**), including `std::vector`, `std::array`, `std::map`, `std::set`, `std::sort`, and modern C++17/20/23 structured bindings.

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_stl_architecture_and_sequence_containers.md` (./01_THEORY/01_stl_architecture_and_sequence_containers.md)
  - `02_associative_containers_map_and_set.md` (./01_THEORY/02_associative_containers_map_and_set.md)
  - `03_iterators_algorithms_and_modern_cpp23.md` (./01_THEORY/03_iterators_algorithms_and_modern_cpp23.md)
- **Programs (`02_PROGRAMS/`)**:

  - `01_vector_and_sort_algorithm.cpp` `(./02_PROGRAMS/01_vector_and_sort_algorithm.cpp)` (Category A)
  - `02_map_frequency_counter.cpp` `(./02_PROGRAMS/02_map_frequency_counter.cpp)` (Category A)
  - `03_vector_at_bounds_check.cpp` `(./02_PROGRAMS/03_vector_at_bounds_check.cpp)` (Category A)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_stl_three_pillars.svg` `(./05_DIAGRAMS/diagram_01_stl_three_pillars.svg)`

01 — STL Architecture and Sequence Containers (`std::vector`, `std::array`)



STL Three Pillars

1. The Three Pillars of STL

1. **Containers**: Store collections of objects (`vector`, `array`, `list`, `map`, `set`).
2. **Iterators**: Generalized pointer-like objects (`begin()`, `end()`) that traverse containers.
3. **Algorithms**: Generic functions (`std::sort`, `std::find`, `std::reverse`) that operate on iterator ranges `[first, last)`.

2. `std::vector<T>`: `size()` vs. `capacity()` and `[]` vs. `.at()`

- `v.size()` is the current number of elements; `v.capacity()` is the total allocated heap slots before reallocation is needed.
- `v[i]` performs **no** bounds check (out-of-bounds is UB); `v.at(i)` performs bounds checking and throws `std::out_of_range` if `i ⩾ v.size()`.

02 — Associative Containers (`std::set` and `std::map`)

- `std::set<T>`: Stores **unique** keys in sorted ascending order ($O(\log n)$ insert/find).
- `std::map<K, V>`: Stores sorted **key-value pairs** with unique keys ($O(\log n)$ lookup).

03 — Iterators, `<algorithm>`, and C++17/20/23 Structured Bindings

CPP

```cpp
for (const auto& [roll, marks] : student_map) {
    cout << roll << " → " << marks << endl;
}
```

C++20/C++23 also provides `std::ranges::sort(vec)` and `.contains(key)` on `std::map` and `std::set`.

Visual Concept & Memory Diagrams (21 STL AND MODERN CPP)

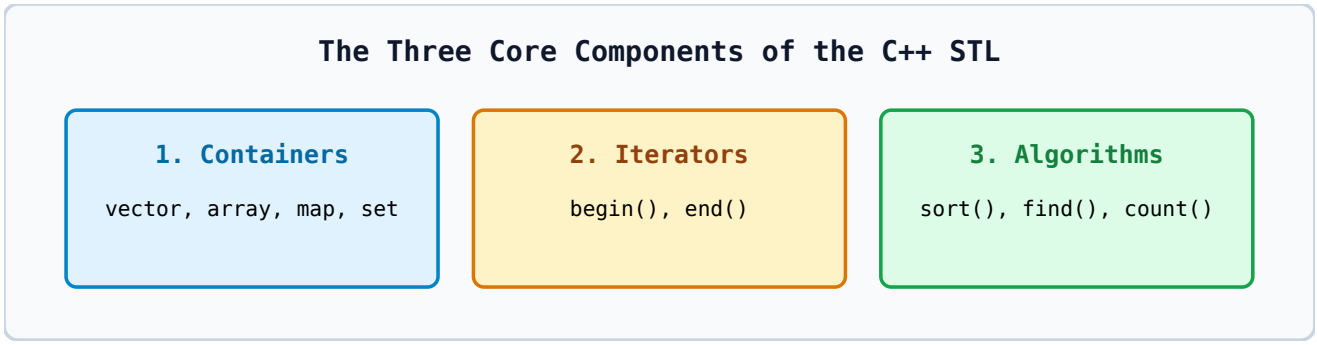


Diagram: diagram 01 stl three pillars

Annotated C++ Programs (21 STL AND MODERN CPP)

Program: `01_vector_and_sort_algorithm.cpp`

C++ SOURCE FILE • 01_vector_and_sort_algorithm.cpp

```cpp
// Program Title: Dynamic Array Using std::vector and Sorting With std::sort
// Topic: STL and Modern C++ — vector and algorithm
// Purpose: Demonstrate push_back(), std::sort(), and range-based for traversal.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_stl_architecture_and_sequence_containers.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> nums = {40, 10, 50, 20, 30};
    nums.push_back(5);

    sort(nums.begin(), nums.end());

    cout << "Sorted vector: ";
    for (int x : nums) {
        cout << x << " ";
    }
    cout << endl;

    return 0;
}

/*
Expected Deterministic Output:
Sorted vector: 5 10 20 30 40 50
*/
```

Program: `02_map_frequency_counter.cpp`

C++ SOURCE FILE • 02_map_frequency_counter.cpp

```cpp
// Program Title: Key-Value Storage Using std::map and Structured Bindings
// Topic: STL and Modern C++ — std::map
// Purpose: Store student roll numbers and marks in a sorted std::map.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_associative_containers_map_and_set.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
#include <map>
using namespace std;

int main() {
    map<int, int> roll_to_marks;
    roll_to_marks[102] = 88;
    roll_to_marks[101] = 95;

    for (const auto& [roll, marks] : roll_to_marks) {
        cout << "Roll " << roll << ": " << marks << endl;
    }

    return 0;
}

/*
Expected Deterministic Output:
Roll 101: 95
Roll 102: 88
*/
```

Program: `03_vector_at_bounds_check.cpp`

C++ SOURCE FILE • 03_vector_at_bounds_check.cpp

```cpp
// Program Title: Safe Bounds Checking With std::vector::at()
// Topic: STL and Modern C++ — Exception-Safe Element Access
// Purpose: Show how v.at(index) throws std::out_of_range on an invalid index.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_stl_architecture_and_sequence_containers.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
#include <vector>
#include <stdexcept>
using namespace std;

int main() {
    vector<int> v = {10, 20, 30};

    try {
        cout << "Element at 1: " << v.at(1) << endl;
        cout << "Element at 10: " << v.at(10) << endl;
    }
    catch (const out_of_range& e) {
        cout << "Caught out_of_range exception safely!" << endl;
    }

    return 0;
}

/*
Expected Deterministic Output:
Element at 1: 20
Caught out_of_range exception safely!
*/
```

Conceptual Questions — Module 21: STL and Modern C++

Q1. Differentiate between `v.size()` and `v.capacity()` in `std::vector`. **Q2.** Differentiate between `v[i]` and `v.at(i)` in `std::vector`.

Debugging Questions — Module 21

Q4. What happens to existing pointers/iterators into a `std::vector` when `push_back()` triggers a capacity reallocation?

Predict the Output Questions — Module 21

Q3. Predict the output of inserting `{30, 10, 20, 10, 30}` into a `std::set<int>` and printing its elements.

Programming Questions — Module 21

Q5. Write a C++23 program that removes duplicates from an integer list and prints elements in ascending order using `std::set`.

Detailed Solutions — Conceptual Questions (Module 21)

- **Q1 Solution**: `size()` is the number of elements currently stored; `capacity()` is the total number of elements the vector can hold in its current heap buffer before reallocating.
- **Q2 Solution**: `v[i]` does not check bounds (Undefined Behavior if out of range); `v.at(i)` checks bounds and throws `std::out_of_range`.

Detailed Solutions — Predict the Output & Debugging (Module 21)

- **Q3 Solution**: `std::set` keeps unique keys in sorted ascending order: `10 20 30`.
- **Q4 Solution**: Reallocation allocates a new heap buffer and frees the old buffer, **invalidating all previous pointers, references, and iterators** into the vector!

Detailed Solutions — Programming Questions (Module 21)

CPP

```cpp
#include <iostream>
#include <set>
using namespace std;
int main() {
    set<int> s = {30, 10, 20, 10, 30};
    for (int x : s) cout << x << " ";
    cout << endl;
    return 0;
}
// Expected Output: 10 20 30
```